

HTTP

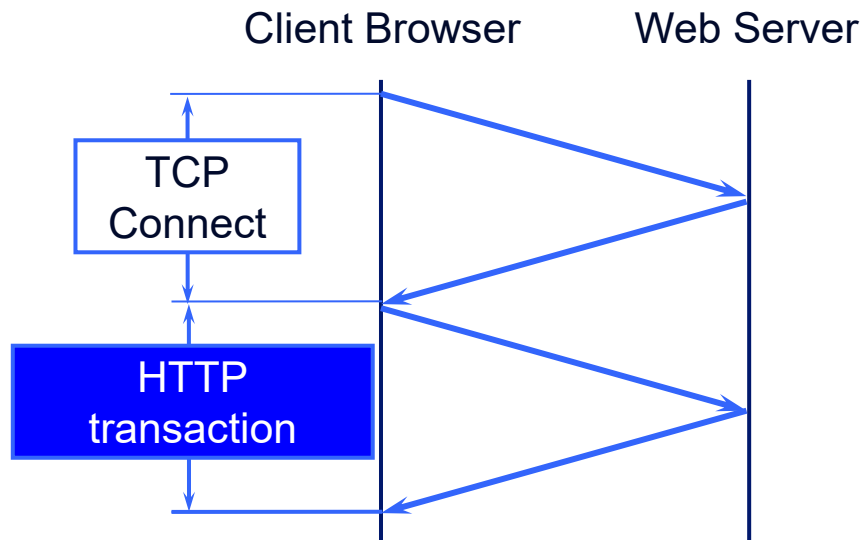
HyperText Transfer Protocol

Miguel Leitão, 2023

HTTP

- HTTP is the protocol that supports communication between Web browsers and Web servers.
- From the RFC: “HTTP is an application-level protocol with the lightness and speed necessary for distributed, hypermedia information systems.”
- The HTTP communication generally takes place over a TCP connection, but the protocol itself is not dependent on a specific transport layer

HTTP Transaction



Request - Response

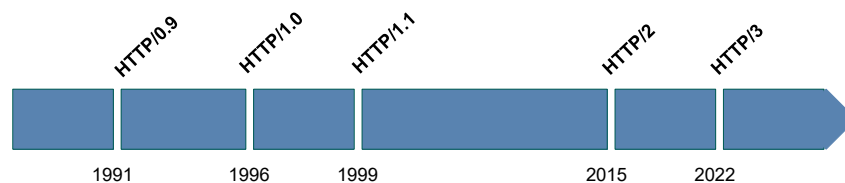
- HTTP has a simple structure:
 - client sends a request
 - server returns a reply.
- HTTP can support multiple request-reply exchanges over a single TCP connection.

Well Known Address

- A “Web Server” is a HTTP server
- The “well known” TCP port for HTTP servers is port 80.
- Other ports can be used as well...

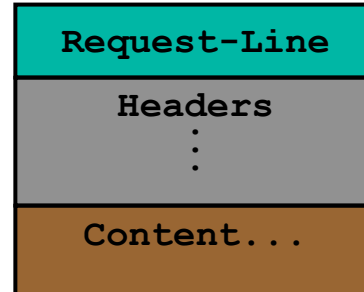
HTTP Versions

- The original version is known as “HTTP Version 0.9”
 - HTTP/0.9 was used for many years.
- Starting with HTTP 1.0 the version number is part of every request.



- HTTP is still changing...

HTTP 1.x Request



- Lines of text (ASCII).
- Lines end with CRLF “\r\n”
- First line is called “Request-Line”

Request Line

Method URI HTTP-Version \r\n

- The request line contains 3 *tokens* (words).
- space characters “ ” separate the tokens.
- Newline (\n) seems to work by itself (but the protocol requires CRLF)

Request Method

The Request Method can be:

GET **HEAD** **PUT**
POST **DELETE** **TRACE**
OPTIONS

future expansion is supported

Methods

- GET: retrieve information identified by the URI.
- HEAD: retrieve meta-information about the URI.
- POST: send information to a URI and retrieve result.

GET, HEAD and POST are supported everywhere.

Methods (other)

- PUT: Store information in location named by URI.
- DELETE: remove *entity* identified by URI.
- TRACE: used to trace HTTP forwarding through proxies, tunnels, etc.
- OPTIONS: used to determine the capabilities of the server, or characteristics of a named resource.

HTTP Version Number

“HTTP/1.0” or “HTTP/1.1”

HTTP 0.9 did not include a version number in a request line.

If a server gets a request line with no HTTP version number, it assumes 0.9

The Header Lines

- After the *Request-Line* come a number (possibly zero) of HTTP *headers*.
- Each header line contains an attribute name followed by a ":" followed by the attribute value.

Headers

- Request Headers provide information to the server about the client
 - what kind of client
 - what kind of content will be accepted
 - who is making the request
 - Web site
- There can be 0 headers

Example HTTP Headers

Accept: text/html

From: neytmann@cybersurg.com

User-Agent: Mozilla/4.0

Referer: http://foo.com/blah

End of the Headers

- Each header ends with a CRLF
- The end of the header section is marked with a blank line.
 - just CRLF
- For GET and HEAD requests, the end of the headers is the end of the request!

Post

- A POST request includes some content (some data) after the headers (after the blank line).
- There is no format for the data (just raw bytes).
- A POST request must include a Content-Length line in the headers:
 - Content-Length: 267

Example GET Request

```
GET /~hollingd/testanswers.html HTTP/1.0
Accept: */*
User-Agent: Internet Explorer
From: cheater@cheaters.org
Referer: http://foo.com/
```

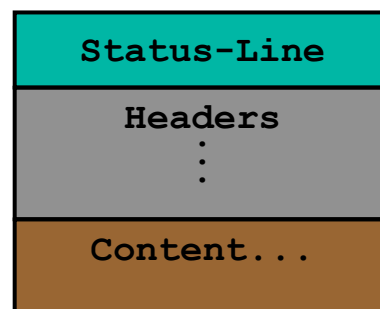
← There is a blank line here!

Example POST Request

```
POST /~hollingd/changegrade.cgi HTTP/1.1
Accept: */*
User-Agent: SecretAgent V2.3
Content-length: 35
Referer: http://monte.cs.rpi.edu/blah

stuid=6660182722&item=test1&grade=99
```

HTTP Response



- ASCII Status Line
- Headers Section
- Content can be anything (not just text)
 - typically is HTML document or some kind of image.

Response Status Line

HTTP-Version Status-Code Message

- *Status Code* is 3 digit number (for computers)
- *Message* is text (for humans)

Examples

HTTP/1.0 200 OK
 HTTP/1.0 301 Moved Permanently
 HTTP/1.0 400 Bad Request
 HTTP/1.0 500 Internal Server Error

Status Codes

1xx Informational
 2xx Success
 3xx Redirection
 4xx Client Error
 5xx Server Error

Response Headers

Provide information about the returned *entity* (document).

- what kind of document
- how big the document is
- how the document is encoded
- when the document was last modified

Example

Date: Wed, 30 Jan 2002 12:48:17 EST
 Server: Apache/1.17
 Content-Type: text/html
 Content-Length: 1756
 Content-Encoding: gzip

Response Header Examples

```
Date: Wed, 30 Jan 2002 12:48:17 EST
Server: Apache/1.17
Content-Type: text/html
Content-Length: 1756
Content-Encoding: gzip
```

Content

- Content can be anything (sequence of raw bytes).
- **Content-Length** header is required for any response that includes content.
- **Content-Type** header also required.

Try it with telnet

```

> telnet www.dee.isep.ipp.pt 80
GET / HTTP/1.0
HTTP/1.0 200 OK
Server: Apache
...

```

Request (points to GET / HTTP/1.0)

Blank Line (end of headers) (points to the blank line between GET and HTTP)

Response (points to HTTP/1.0 200 OK)

Single Request/Reply

- The client sends a complete request.
- The server sends back the entire reply.
- The server closes it's socket.
- If the client needs another document it must open a new connection.

Persistent Connections

- HTTP 1.1 supports persistent connections (this is supposed to be the default).
- Multiple requests can be handled.
- Most servers seem to close the connection after the first response...

HTTP 1.1 Head Request

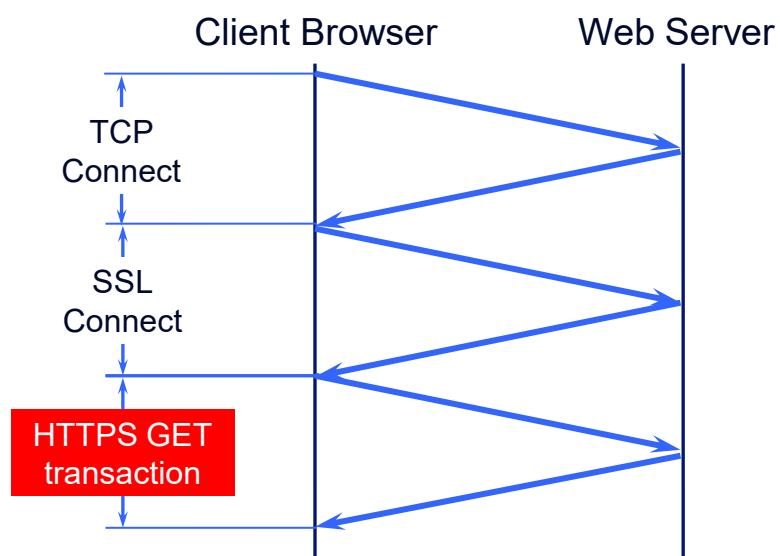
```
$ telnet linuxzoo.net 80
HEAD / HTTP/1.1
Host: tiger.net

HTTP/1.1 200 OK
Date: Mon, 01 Nov 2008 15:06:44 GMT
Server: Apache/2.0.46 (Red Hat)
Last-Modified: Fri, 29 Oct 2008 14:47:22 GMT
ETag: "4981dd-920-22ea7280"
Accept-Ranges: bytes
Content-Length: 2336
Content-Type: text/html; charset=UTF-8
```

HTTPS

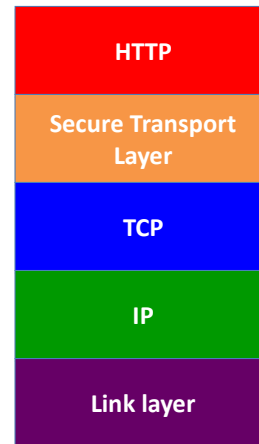


HTTPS Transaction



HTTP-S: Securing HTTP

- HTTP sits on top of secure channel (SSL/TLS)
 - https:// vs. http://
 - TCP port 443 vs. 80
- All (HTTP) bytes encrypted and authenticated
 - No change to HTTP itself!



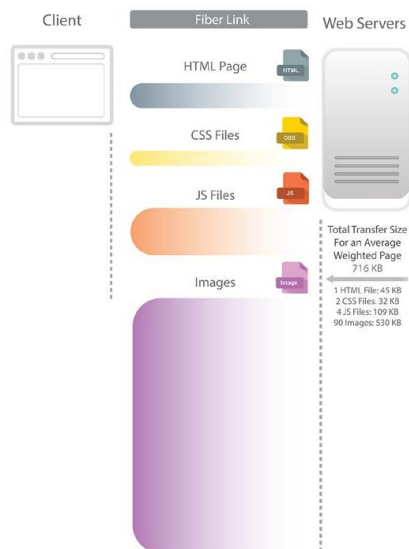
31

Hierarchical Public Key Infrastructure

- **Public key certificate**
 - Binding between identity and a public key
 - “Identity” is, for example, a domain name
 - Digital signature to ensure integrity
- **Certificate authority**
 - Issues public key certificates and verifies identities
 - Trusted parties (e.g., VeriSign, GoDaddy, Comodo)
 - Preconfigured certificates in Web browsers

32

Typical HTTP use



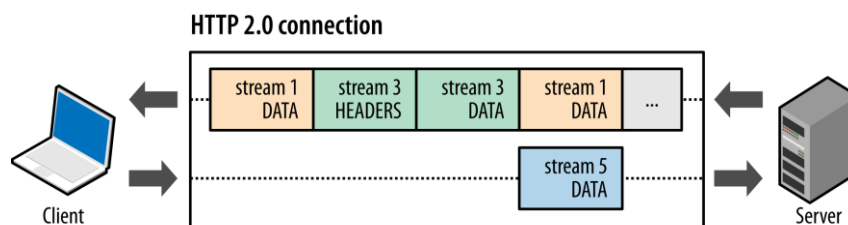
- A Web page is set of many items.
- Items are known after parsing HTML doc.
- Each item is downloaded separately.
- Items from the same server are downloaded sequentially.

HTTP/2

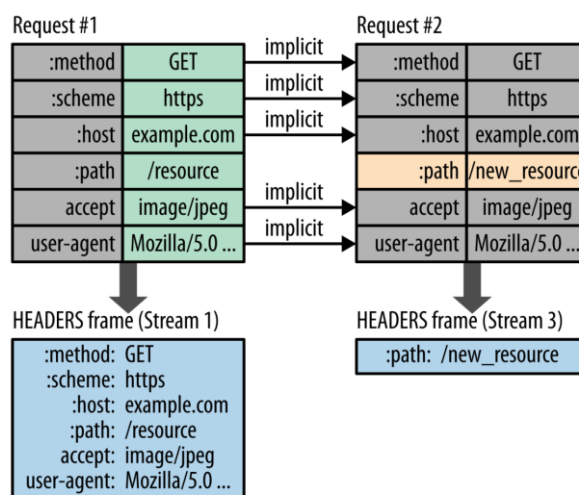


- First major update since HTTP/1.1
- **Binary**, instead of textual.
- Fully **multiplexed**
 - Allows sending multiple requests in parallel over a single TCP connection.
- Uses header compression **HPACK** to reduce overhead.
- Allows servers to **PUSH** responses to clients.
- Uses the new **ALPN** extension which allows for faster encrypted connections
- **Domain sharding** and **asset concatenation** are no longer needed.

Connection

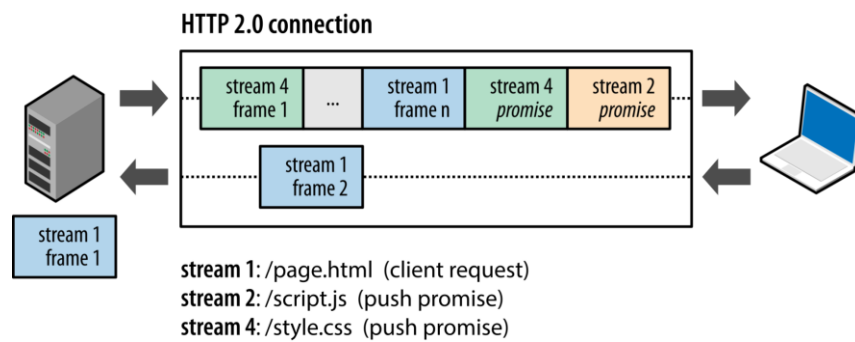


HPACK



Push

1 Request / n Responses



PUSH

